Java Backend Architecture

Interview Series

Chapter 18.7

Nginx as Kubernetes Ingress

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 18.7 – Nginx as Kubernetes Ingress

Overview

The Nginx Ingress Controller is the most widely deployed Kubernetes Ingress implementation, managing external HTTP/HTTPS access to Java services running in a cluster. It translates Kubernetes Ingress resources into Nginx configuration, supporting path-based routing, TLS termination, canary releases, authentication, and rate limiting via annotations. Understanding its architecture and annotation system is essential for Java backend deployments.

Key Definitions

Term	Definition
Ingress Controller	A Kubernetes controller that watches Ingress objects and configures Nginx accordingly.
Ingress resource	Kubernetes object defining routing rules: host + path -> Service:port.
IngressClass	Kubernetes object specifying which controller handles an Ingress. Multiple controllers can coexist.
Annotations	Nginx-specific configuration on Ingress objects (nginx.ingress.kubernetes.io/*).
ConfigMap	Global Nginx controller configuration affecting all Ingresses (proxy timeouts, log format, etc.).
canary	Annotation-based traffic splitting: route a percentage of traffic to canary Java deployment.
rewrite-target	Annotation to strip path prefix before forwarding to backend Java service.
auth-url	Annotation pointing to an external auth service for request validation (JWT, OAuth2).
Gateway API	Next-generation replacement for Ingress: HTTPRoute, GatewayClass, Gateway resources.
ModSecurity annotation	Enable WAF per Ingress: nginx.ingress.kubernetes.io/enable-modsecurity: 'true'.

Diagram 1: Nginx Ingress Controller Architecture

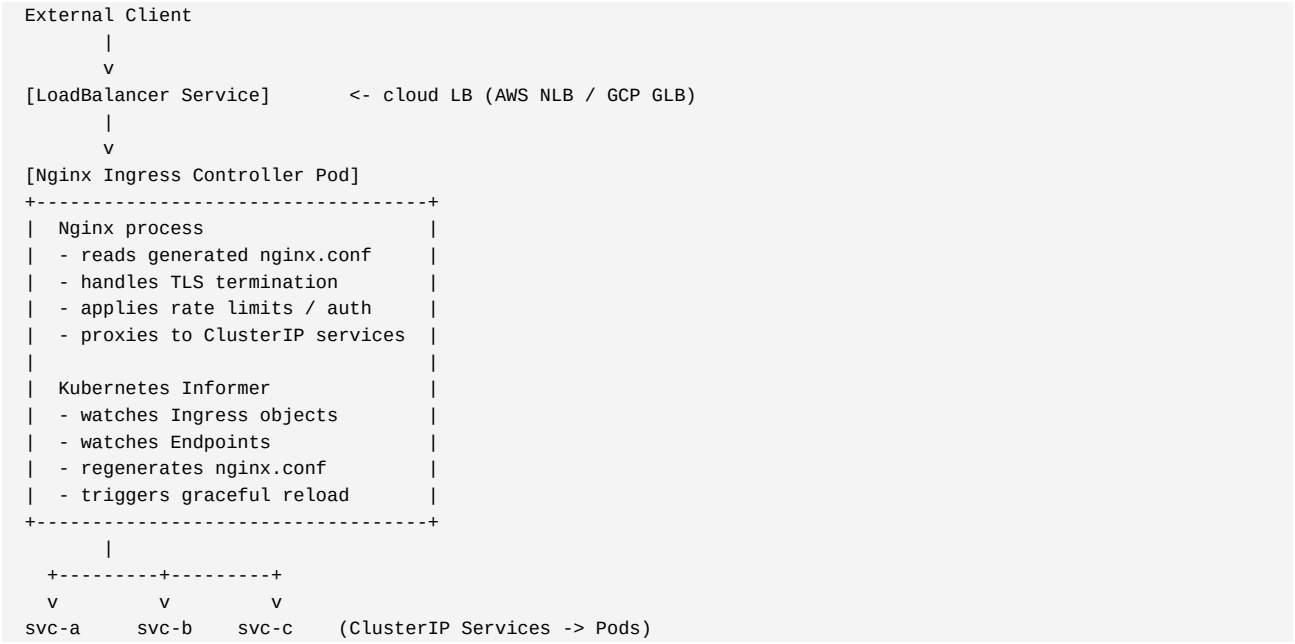
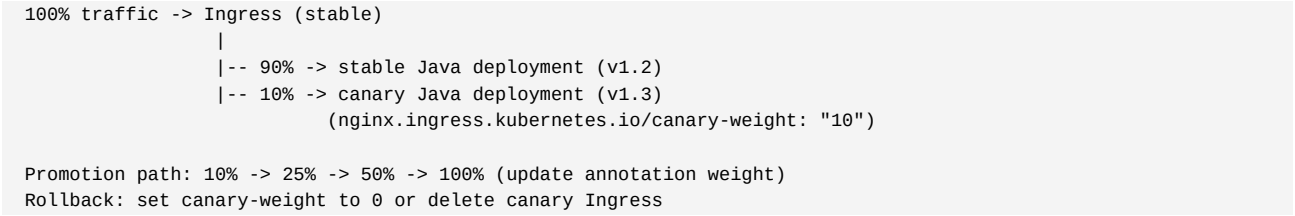


Diagram 2: Canary Deployment Traffic Split



Code Examples

Basic Ingress Resource:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: java-api
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/proxy-body-size: 10m
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "5"
spec:
  ingressClassName: nginx
  tls:
    - hosts: [api.example.com]
      secretName: api-tls
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /api(/|$)(.*)
            pathType: Prefix
            backend:
              service: { name: java-api-svc, port: { number: 8080 } }
```

Rate Limiting + Auth Annotation:

```
metadata:
  annotations:
    # Rate limit: 10 r/s per IP
    nginx.ingress.kubernetes.io/limit-rps: "10"
    nginx.ingress.kubernetes.io/limit-burst-multiplier: "5"

    # External auth (JWT validation)
    nginx.ingress.kubernetes.io/auth-url: "http://auth-service.auth.svc/validate"
    nginx.ingress.kubernetes.io/auth-method: GET
    nginx.ingress.kubernetes.io/auth-response-headers: X-User-Id, X-Roles

    # CORS
    nginx.ingress.kubernetes.io/enable-cors: "true"
    nginx.ingress.kubernetes.io/cors-allow-origin: "https://app.example.com"
    nginx.ingress.kubernetes.io/cors-allow-methods: "GET, POST, PUT, DELETE"
```

Canary Deployment:

```
# Stable Ingress (already exists)
---
# Canary Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: java-api-canary
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10"  # 10% to canary
    # Alternative: cookie-based: canary-by-cookie: "X-Canary"
    # Alternative: header-based: canary-by-header: "X-Canary-Version"
spec:
  ingressClassName: nginx
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
```

```
            service: { name: java-api-canary-svc, port: { number: 8080 } }
```

Interview Q&A – Nginx as Kubernetes Ingress

Q1. How does the Nginx Ingress Controller translate Ingress to nginx.conf?

The controller runs a Kubernetes informer watching Ingress and Endpoint objects. When objects change, it renders a new nginx.conf using Go templates, validates with nginx -t, and triggers nginx -s reload for graceful configuration update without dropping connections. Endpoints (pod IPs) are embedded directly in upstream blocks, bypassing kube-proxy for better performance and accurate least_conn tracking.

Q2. What is rewrite-target and why is it needed?

rewrite-target: /$2 strips the path prefix. path: /api(/|$)(.*) + rewrite-target: /$2 means /api/users -> /users on the backend. Without rewrite-target, the Java service receives /api/users and must handle the prefix. Using capture group $2 extracts the path after the prefix. The capture group numbering matches PCRE groups in the path regex.

Q3. How does canary work and what are its routing strategies?

Canary Ingress: nginx.ingress.kubernetes.io/canary: 'true' marks it as canary. Routing strategies: canary-weight (percentage of all traffic), canary-by-header (route if header matches value, e.g. X-Canary: always), canary-by-cookie (route based on cookie value). Weight-based is standard. Header/cookie enables targeted testing (internal team always gets canary).

Q4. What is the difference between Ingress and Gateway API?

Ingress: simple, widely supported, but limited (no traffic splitting, no header modification, annotations are non-standard). Gateway API: Kubernetes-native, role-based (GatewayClass for infra, Gateway for cluster ops, HTTPRoute for developers), supports traffic weighting natively, header manipulation, URL rewrite without annotations. Nginx Ingress Controller supports both. Gateway API is the strategic direction.

Q5. How does the controller handle TLS certificates?

spec.tls references a Kubernetes Secret (kubernetes.io/tls type) containing tls.crt and tls.key. The controller mounts the Secret as Nginx ssl_certificate/ssl_certificate_key. cert-manager integration: Ingress annotation cert-manager.io/cluster-issuer triggers automatic certificate provisioning from Let's Encrypt. Secret rotation: controller detects Secret change via informer and reloads Nginx.

Q6. How do you configure global Nginx settings for all Ingresses?

ConfigMap nginx-configuration in ingress-nginx namespace. Keys map to Nginx directives: proxy-read-timeout: '60', use-forwarded-headers: 'true', worker-processes: 'auto', log-format-upstream: JSON format string. Changes trigger controller config reload. Per-Ingress settings via annotations override global ConfigMap values for that Ingress.

Q7. How does auth-url implement external authentication?

auth-url annotation: controller generates auth_request /auth-validate location calling the specified URL. The auth service receives the request (same headers, no body). 2xx: request proceeds. 401/403: client gets that status. auth-response-headers: X-User-Id, X-Roles -- auth service response headers passed to Java backend as request headers. Centralises auth for all Java services behind the Ingress.

Q8. How do you handle WebSocket connections via Nginx Ingress?

Nginx Ingress supports WebSocket upgrades automatically when proxy-http-version: '1.1' and proxy-read-timeout is set appropriately (e.g. '3600'). The controller sets proxy_set_header Upgrade and Connection upgrade headers. For Spring WebSocket / SockJS: ensure the service port is the same as HTTP and the path is included in Ingress rules.

Q9. What are the performance implications of Nginx Ingress Controller reloads?

Each nginx -s reload starts new worker processes -- brief CPU spike. With many Ingress objects or frequent Endpoint changes (pod scaling), reloads can be frequent. Mitigation: nginx-ingress uses debouncing (groups changes within a short window into one reload). Nginx Plus supports dynamic upstream reconfiguration without reload. Use kubectl scale carefully -- each pod scale event triggers an Endpoint change and reload.

Q10. How do you expose metrics from Nginx Ingress Controller?

Controller exposes Prometheus metrics at :10254/metrics. Key metrics: nginx_ingress_controller_requests (request count by status/ingress/service), nginx_ingress_controller_ingress_upstream_latency_seconds (p50/p95/p99 per backend), nginx_ingress_controller_ssl_expire_time_seconds (cert expiry monitoring). Deploy ServiceMonitor (Prometheus Operator) to scrape. Alert on high 5xx rates and cert expiry within 30 days.